Sandya Mannarswamy

CODE SPORT

In this month's column, the discussion on the features of JavaScript continues. For the last couple of months, what has been covered is dynamic languages such as JavaScript, and how they differ from traditional statically compiled languages like C/C++. In this month's column, the focus is on an important feature of JavaScript—closures.

In last month's column, we had looked at a few examples of closures. The official Wikipedia entry defines a closure as "…a function together with a referencing environment for the non-local variables of that function." Well, while this definition does in fact cover closures, it fails to distinguish how closures are different from, say, function pointers in C/C++. After all, a function has a referencing environment for globals in C/C++, and can be passed around through a function pointer—so does that constitute a closure? No, not really. Let us consider the example below:

```
var count = 20;
function incrementCount(int incr) {
        return count + incr;
}
assert( incrementCount(5) == 25);
```

Here, the function call to *incrementCount()* references the variable *count*, and hence the returned value is 25. Consider a slight modification of the above code:

```
var count = 20;
function incrementCount(int incr) {
        return count + incr;
}
count = 30;
assert( incrementCount(5) == 25);
```

Now the assert actually fails, and the return value from *'incrementCount '* is actually 35, not 25. That is because the reference to the global variable *'count '* returns the current value of the variable *'count '*, and not its value from the lexical scope reference of *'count '* in the function *'incrementCount '*. Hence, the above two examples are not really closures. Therefore, a slightly improved definition would be that *a closure is a function that has the following properties*:

- It can be passed around like an object.
- It 'remembers' all variables that were in scope when it was created, so that these can be referenced by the called closure even though the parent function where the variables were defined is no longer in scope.

The second property is the most difficult thing for a C/C++ programmer to understand. The question that crops up is: How can the closure function access the variables of the enclosing function when the latter has already returned, and its local variables have gone out of scope? Well, the answer to that is that JavaScript supports closures as first-class citizens, hence as long as there is a reference to the enclosing function's locals by a closure function, the JavaScript runtime will not free up the local variables of the enclosing function. Hence, these variables remain accessible to the closure function, even though the execution of the enclosing function has completed.

Closures need to be supported as a feature of the language. Languages such as LISP, Scheme